

MySQL Stored Procedures — Deep Notes

Language: Tanglish (Tamil + English) | **Purpose:** Study / Interview / Practical Development

Core definition: MySQL Stored Procedure is a named, reusable set of SQL statements and procedural logic stored inside the database and executed using **CALL**.

1. Stored Procedure na enna?

Stored Procedure = database-kulla save panni vachirukkura reusable SQL statements-oda group.
Application-la same database logic repeated-a use panna, procedure create panni CALL pannalam.

```
CREATE PROCEDURE getAllStudents()  
BEGIN  
  SELECT * FROM students;  
END;  
  
CALL getAllStudents();
```

2. Why Stored Procedure?

Stored Procedures use panna main reasons: reusable logic, centralized database logic, complex multi-step operations, transactions, error handling, and controlled database access.

Without Procedure: Laravel/PHP → SQL Query → MySQL

With Procedure: Laravel/PHP → CALL procedure → MySQL Procedure → SQL → Tables

3. Basic Syntax

```
DELIMITER //
```

```
CREATE PROCEDURE procedure_name()  
BEGIN  
  -- SQL statements  
END //
```

```
DELIMITER ;
```

CREATE PROCEDURE creates the procedure. BEGIN...END contains the statements. CALL executes the procedure.

4. DELIMITER

MySQL client normally uses semicolon (;) to identify the end of a statement. A procedure contains many internal statements ending with ;, so DELIMITER is temporarily changed to something like //.

```
DELIMITER //
```

```
CREATE PROCEDURE getAllStudents()  
BEGIN  
  SELECT * FROM students;  
  SELECT COUNT(*) FROM students;  
END //
```

```
DELIMITER ;
```

Important: DELIMITER is mainly a client-side instruction; it is not stored as part of the procedure logic.

5. BEGIN ... END

BEGIN and END define a block containing multiple SQL/procedural statements.

```
BEGIN
INSERT INTO students ...;
UPDATE courses ...;
SELECT * FROM students;
END
```

6. Procedure with IN Parameter

```
DELIMITER //

CREATE PROCEDURE getStudentById(IN studentId INT)
BEGIN
SELECT *
FROM students
WHERE id = studentId;
END //

DELIMITER ;

CALL getStudentById(2);
```

IN means caller sends a value into the procedure. Think of it similar to a PHP function parameter.

7. IN / OUT / INOUT

IN: input only. **OUT**: procedure produces a value into a caller variable. **INOUT**: receives a value, modifies it, and returns the modified value.

```
-- OUT example
DELIMITER //

CREATE PROCEDURE getStudentCount(OUT total INT)
BEGIN
SELECT COUNT(*) INTO total
FROM students;
END //

DELIMITER ;

CALL getStudentCount(@total);
SELECT @total;

-- INOUT example
DELIMITER //

CREATE PROCEDURE increaseNumber(INOUT num INT)
BEGIN
SET num = num + 10;
END //

DELIMITER ;

SET @number = 20;
CALL increaseNumber(@number);
SELECT @number;
```

8. Local Variables — DECLARE

Procedure-kulla temporary/local variable create panna DECLARE use pannuvom.

```
DELIMITER //
```

```
CREATE PROCEDURE studentExample()  
BEGIN  
  DECLARE totalStudents INT;  
  
  SELECT COUNT(*)  
  INTO totalStudents  
  FROM students;  
  
  SELECT totalStudents;  
END //
```

```
DELIMITER ;
```

DECLARE usually comes at the beginning of the BEGIN...END block before executable statements.

9. SET

```
DECLARE x INT;  
SET x = 100;  
SELECT x;
```

SET is used to assign or change a variable's value.

10. IF / ELSEIF / ELSE

```
DELIMITER //
```

```
CREATE PROCEDURE checkStudentAge(IN studentAge INT)  
BEGIN  
  IF studentAge >= 18 THEN  
    SELECT 'Adult' AS status;  
  ELSE  
    SELECT 'Minor' AS status;  
  END IF;  
END //
```

```
DELIMITER ;
```

Stored Procedures support programming-style conditional logic.

11. CASE

```
CASE  
  WHEN marks >= 90 THEN  
    SET grade = 'A';  
  WHEN marks >= 75 THEN  
    SET grade = 'B';  
  WHEN marks >= 50 THEN  
    SET grade = 'C';  
  ELSE  
    SET grade = 'F';  
END CASE;
```

12. Loops

12.1 WHILE

```

WHILE counter <= 5 DO
  SELECT counter;
  SET counter = counter + 1;
END WHILE;

```

12.2 LOOP + LEAVE

```

myLoop: LOOP
  SET counter = counter + 1;

  IF counter > 5 THEN
    LEAVE myLoop;
  END IF;
END LOOP;

```

LEAVE is used to exit a labeled loop.

13. Multiple Database Operations

A procedure can combine several operations into one reusable database operation.

```

CREATE PROCEDURE registerStudent(...)
BEGIN
  INSERT INTO students ...;
  INSERT INTO student_courses ...;
  UPDATE courses
  SET enrolled_count = enrolled_count + 1;
  INSERT INTO audit_log ...;
END;

```

Application can call one procedure instead of repeating all the SQL logic.

14. Transactions

Transactions are important when multiple operations must succeed or fail together.

```

START TRANSACTION;

INSERT INTO students ...;
INSERT INTO student_courses ...;
UPDATE courses ...;

COMMIT;

```

If something fails, ROLLBACK can undo the transaction.

```

START TRANSACTION;

-- operations

ROLLBACK;

```

COMMIT = save transaction. **ROLLBACK** = undo transaction.

15. Error Handling — HANDLER

```

DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
  ROLLBACK;
END;

```

This pattern can be used so that SQL exceptions trigger cleanup such as ROLLBACK. In production, the exact error-handling strategy should be designed carefully.

16. Real-world Enrollment Example

Student enrollment flow could be: 1) check student exists, 2) check course exists, 3) check duplicate enrollment, 4) insert student_course, 5) update course count, 6) insert audit record.

```
CREATE PROCEDURE enrollStudent(
  IN p_student_id INT,
  IN p_course_id INT
)
BEGIN
  -- validation
  -- insert enrollment
  -- update count
  -- audit
END;
```

Laravel can call it using a parameterized CALL statement.

17. Laravel + Stored Procedure

```
$result = DB::select(
  'CALL getStudentById(?)',
  [$id]
);
```

Flow: Browser → Laravel Controller → DB::select() → CALL procedure → MySQL executes procedure → tables → result → Laravel.

18. Stored Procedure vs PHP Function

PHP function: application-side logic. **Stored Procedure:** database-side executable logic.

```
PHP:
function getStudent($id) {
  // application code
}

MySQL:
CREATE PROCEDURE getStudent(IN id INT)
BEGIN
  SELECT * FROM students WHERE students.id = id;
END;
```

19. Stored Procedure vs View

View: mainly a saved SELECT/query representation that can be queried like a table. **Procedure:** executable database logic that can use parameters, variables, conditions, loops, DML, transactions, and handlers.

20. Stored Procedure vs Function

A MySQL stored function returns a value and is generally used in SQL expressions. A procedure is invoked with CALL and can perform broader procedural operations.

```
-- Function style
SELECT addNumbers(10, 20);
```

```
-- Procedure style  
CALL getStudent(10);
```

21. Advantages

- Reusability — same logic can be called multiple times.
- Centralized database logic.
- Can handle multi-step operations.
- Supports transactions and error handling.
- Can reduce repeated application-side SQL.
- Can be useful for database-level access control.

22. Disadvantages

- MySQL-specific logic can increase database vendor dependency.
- Business logic may become harder to discover if spread across many procedures.
- Testing and deployment require database-aware workflows.
- Putting too much application logic into procedures can hurt maintainability.

23. Important Commands — Quick Revision

Concept	Meaning
CREATE PROCEDURE	Create/save a procedure
CALL	Execute a procedure
BEGIN...END	Procedure statement block
DELIMITER	Changes client statement delimiter temporarily
IN	Input parameter
OUT	Output parameter
INOUT	Input + output parameter
DECLARE	Create local variable
SET	Assign/change variable value
IF	Conditional logic
CASE	Multiple conditional branches
WHILE / LOOP	Looping
START TRANSACTION	Begin transaction
COMMIT	Save transaction
ROLLBACK	Undo transaction
HANDLER	Handle SQL conditions/errors

24. Study Order

Recommended order: 1) CREATE PROCEDURE + CALL → 2) IN/OUT/INOUT → 3) DECLARE + variables → 4) IF/ELSEIF/ELSE → 5) CASE → 6) WHILE/LOOP/LEAVE → 7) SELECT INTO → 8) Transactions → 9) HANDLER → 10) Laravel integration → 11) Procedure vs Function/Trigger/View → 12) Real-world CRUD/enrollment project.

25. One-line Definition for Interview

MySQL Stored Procedure is a named, reusable set of SQL statements and procedural logic stored inside the database and executed using CALL.